

System Architecture

1. Architectural Goals and Constraints

- A multi-tier web application reachable from a browser on desktop, tablet, or phone.
- Role-based security (RBAC) enforced across every subsystem.
- Strong data integrity for stock levels and account balances, so concurrent sales and deliveries stay consistent and stock never goes negative.
- A single store and a single currency, so distribution, multi-tenancy, and legacy or ESB integration are not needed in this release.
- Clear separation of layers and subsystems, to keep the system maintainable and to support the later OO design work.

2. Architecturally Significant Requirements

These are the requirements that shape the architecture:

- **Record Sale and Record Delivery** both change stock. They drive a transactional data-access layer and a business layer that enforces "no negative stock" and updates balances atomically.
- **RBAC across all subsystems** drives an authentication and authorization mechanism at the API boundary.
- **Business-rule validation** (unit breakdown must be a positive integer, manager-only pricing, supplier payment terms) drives a centralized Rules Engine component in the business layer.
- **Persistence** of products, stock, purchase orders, invoices, and expenses drives a data-access layer over a relational database through an ORM.

3. Chosen Architecture

A front-end single-page app built with React and Typescript with support for local persistence and data caching to feel fast to interact with.

A back-end REST API service. Within the back-end, requests flow down through a API layer, a Business Service layer, a Data Access layer, and the ORM, to the database.

